

Explicação dos Exercícios de Lógica

Exercício 68 - Gerador de Personagem de D&D | Exercício 69 - Grau de Separação

Soluções em PHP, desenvolvidas a partir de pseudocódigo em Portugol.

Exercício 68 — Gerador de Personagem de D&D

O que o exercício pede

Em Dungeons & Dragons, um personagem tem seis atributos: força, destreza, constituição, inteligência, sabedoria e carisma. Cada atributo é sorteado jogando 4 dados de 6 lados, descartando o menor valor e somando os outros 3. Os pontos de vida do personagem são 10 mais o "modificador de constituição", que é calculado assim: subtrai-se 10 da constituição, divide-se por 2 e arredonda-se para baixo (não para o zero — para baixo mesmo, o que importa quando o resultado é negativo).

Como o problema foi dividido

Em vez de escrever tudo dentro de uma função só, a solução final ficou separada em três métodos estáticos da classe `DndCharacter`, porque os testes automatizados do exercício chamam cada um deles de forma independente:

- `modifier(constituicao)` — recebe um número e devolve só o cálculo do modificador.
- `ability()` — faz uma única jogada de atributo (4 dados, descarta o menor, soma os 3 maiores).
- `generate()` — monta um personagem completo, chamando `ability()` seis vezes e calculando os pontos de vida.

Passo a passo do código

Passo 1 — `modifier()`

Recebe a constituição e devolve o modificador. A divisão em PHP (`/`) já produz um número com casas decimais quando o resultado não é exato (por exemplo, $-7 / 2$ vira -3.5), diferente de outras linguagens que truncam. A função `floor()` arredonda esse valor para baixo (-3.5 vira -4.0), e o `(int)` na frente converte o resultado de volta para um número inteiro.

```
public static function modifier(int $constituicao): int
{
    return (int) floor(($constituicao - 10) / 2);
}
```

Passo 2 — `ability()`

Joga 4 dados (`random_int(1, 6)` simula um dado de 6 lados). Depois, percorre os 4 valores para descobrir qual é o menor e guarda a posição dele em `$indiceParaOMenor`. Por fim, soma todos os dados exceto o que está naquela posição — ou seja, soma os 3 maiores.

```
public static function ability(): int
{
    $dados = [];

    for ($j = 0; $j < 4; $j++) {
        $dados[$j] = random_int(1, 6);
    }
}
```

```

$menor = $dados[0];
$indiceParaOMenor = 0;

for ($y = 0; $y < 4; $y++) {
    if ($dados[$y] < $menor) {
        $menor = $dados[$y];
        $indiceParaOMenor = $y;
    }
}

$soma = 0;

for ($x = 0; $x < 4; $x++) {
    if ($x != $indiceParaOMenor) {
        $soma = $soma + $dados[$x];
    }
}

return $soma;
}

```

Passo 3 — generate()

Cria um novo objeto DndCharacter (new self()) e preenche cada atributo chamando ability() uma vez para cada um. Repara que, dentro de um método estático, chama-se outro método estático da mesma classe com self:: (e não \$this->). Por fim, calcula os pontos de vida usando modifier() sobre a constituição já sorteada, e devolve o personagem pronto.

```

public static function generate(): self
{
    $personagem = new self();

    $personagem->strength = self::ability();
    $personagem->dexterity = self::ability();
    $personagem->constitution = self::ability();
    $personagem->intelligence = self::ability();
    $personagem->wisdom = self::ability();
    $personagem->charisma = self::ability();

    $personagem->hitpoints = 10 + self::modifier($personagem->constitution);

    return $personagem;
}

```

Código completo do exercício 68

```
<?php

declare(strict_types=1);

class DndCharacter
{
    public function __construct()
    {

    }

    public static function modifier(int $constituicao): int
    {
        return (int) floor(($constituicao - 10) / 2);
    }

    public static function ability(): int
    {
        $dados = [];

        for ($j = 0; $j < 4; $j++) {
            $dados[$j] = random_int(1, 6);
        }

        $menor = $dados[0];
        $indiceParaOMenor = 0;

        for ($y = 0; $y < 4; $y++) {
            if ($dados[$y] < $menor) {
                $menor = $dados[$y];
                $indiceParaOMenor = $y;
            }
        }

        $soma = 0;

        for ($x = 0; $x < 4; $x++) {
            if ($x != $indiceParaOMenor) {
                $soma = $soma + $dados[$x];
            }
        }

        return $soma;
    }

    public static function generate(): self
    {
        $personagem = new self();

        $personagem->strength = self::ability();
        $personagem->dexterity = self::ability();
        $personagem->constitution = self::ability();
        $personagem->intelligence = self::ability();
        $personagem->wisdom = self::ability();
        $personagem->charisma = self::ability();

        $personagem->hitpoints = 10 + self::modifier($personagem->constitution);

        return $personagem;
    }
}
```

Exercício 69 — Grau de Separação (árvore genealógica)

O que o exercício pede

Dada uma árvore genealógica (quem é pai/mãe de quem), a função `degreeOfSeparation` deve dizer quantas "conexões" existem entre duas pessoas — por exemplo, um pai e seu filho têm grau 1, um avô e seu neto têm grau 2. Se não houver nenhum caminho entre as duas pessoas, a função deve devolver -1.

Formato dos dados de entrada

O parâmetro `$familyTree` é um array associativo em que cada chave é o nome de um pai/mãe, e o valor é a lista dos filhos dessa pessoa. Por exemplo:

```
$familyTree = [  
  "Helena" => ["Isla", "Tariq"],  
  "Isla"   => ["Uma"],  
];
```

A ideia do algoritmo: busca em largura (BFS)

O problema é resolvido tratando a árvore genealógica como um grafo, onde cada ligação pai-filho é uma "aresta" que pode ser percorrida nos dois sentidos (de pai para filho e de filho para pai). Para achar o caminho mais curto entre duas pessoas nesse grafo, usa-se uma técnica chamada busca em largura (Breadth-First Search, ou BFS).

A ideia da BFS é explorar o grafo "camada por camada": primeiro visita quem está a 1 passo de distância da origem, depois quem está a 2 passos, e assim por diante. Isso garante que, assim que a pessoa procurada é encontrada, a distância registrada é garantidamente a menor possível — não existe um caminho mais curto que ainda não tenha sido descoberto.

Para isso, o algoritmo usa uma fila (uma lista em que se adiciona no final e se retira do início) e uma lista de "visitados", para não processar a mesma pessoa duas vezes.

Passo a passo do código

Passo 1 — Preparar a fila com o ponto de partida

A busca começa com `$personA` (a origem) já dentro da fila, na distância 0. O array `$visitados` serve para nunca colocar a mesma pessoa na fila duas vezes.

```
$fila = [];  
$distancia = [];  
$visitados = [];  
  
$tamanhoFila = 0;  
$inicioFila = 0;  
$tamanhoVisitados = 0;  
  
$fila[0] = $personA;  
$distancia[0] = 0;  
$tamanhoFila = 1;  
  
$visitados[0] = $personA;  
$tamanhoVisitados = 1;
```

Passo 2 — O laço principal: percorrer a fila

\$inicioFila funciona como um "ponteiro" que indica qual é a próxima pessoa a processar. Enquanto ainda houver gente na fila (`inicioFila < tamanhoFila`), tira-se a pessoa da vez (`atual`) e a distância dela até a origem. Se essa pessoa for exatamente quem estamos procurando (`personB`), a resposta é a distância dela — e como a BFS explora por camadas, essa é garantidamente a menor distância possível.

```
while ($inicioFila < $tamanhoFila) {
    $atual = $fila[$inicioFila];
    $distanciaAtual = $distancia[$inicioFila];
    $inicioFila = $inicioFila + 1;

    if ($atual == $personB) {
        return $distanciaAtual;
    }

    // (próximo passo: procurar quem está ligado a "atual")
}

return -1;
```

Um detalhe importante: irmãos contam como 1 grau, não 2

Se a "vizinhança" de cada pessoa fosse apenas pai e filhos, dois irmãos (filhos do mesmo pai/mãe) ficariam a distância 2 um do outro: irmão A -> pai -> irmão B. Mas o exercício define que irmãos têm grau de parentesco 1, seguindo a mesma convenção usada em genealogia real (irmãos são "parentes de 1º grau", assim como pai e filho). O mesmo vale, em cascata, para qualquer parentesco que não seja em linha direta (primos, tios, sobrinhos etc.): o caminho mais curto entre duas pessoas assim sempre "sobe até um ancestral comum e desce de novo", e esse único ponto de virada (subir + descer) deve contar como 1 conexão, não 2.

A solução é simples: além de ligar cada pessoa ao seu pai/mãe e aos seus filhos, também se liga diretamente cada pessoa aos seus irmãos (outros filhos do mesmo pai/mãe). Assim, o "atalho" fica embutido no próprio grafo, e a busca em largura encontra automaticamente o caminho mais curto correto, seja qual for o grau de parentesco.

Passo 3 — Encontrar os vizinhos da pessoa atual

Percorre-se `$familyTree` com `foreach`, onde `$pai` é a chave (o nome do pai/mãe) e `$filhos` é a lista de filhos dessa pessoa. Há duas situações possíveis para "atual": ou ela é o próprio pai/mãe listado (nesse caso, os filhos dele são vizinhos), ou ela aparece dentro da lista de filhos de outra pessoa — nesse caso, tanto o pai/mãe quanto os demais filhos dessa lista (os irmãos de "atual") viram vizinhos. A função `in_array()` verifica se um valor está dentro de um array.

```
foreach ($familyTree as $pai => $filhos) {
    $vizinhos = [];

    if ($pai == $atual) {
        $vizinhos = array_merge($vizinhos, $filhos);
    }

    if (in_array($atual, $filhos)) {
        $vizinhos[] = $pai;

        foreach ($filhos as $irmao) {
            if ($irmao != $atual) {
                $vizinhos[] = $irmao;
            }
        }
    }
}
```

```

        }
    }

    // (próximo passo: adicionar cada vizinho novo na fila)
}

```

Passo 4 — Adicionar vizinhos novos na fila

Para cada vizinho encontrado, verifica-se se ele já está em \$visitados. Se ainda não estiver, ele entra na lista de visitados e também na fila, com a distância da pessoa atual mais 1 — porque ele está exatamente um passo mais longe da origem do que "atual".

```

foreach ($vizinhos as $vizinho) {
    $jaVisitado = false;
    for ($j = 0; $j < $tamanhoVisitados; $j++) {
        if ($visitados[$j] == $vizinho) {
            $jaVisitado = true;
        }
    }

    if ($jaVisitado == false) {
        $visitados[$tamanhoVisitados] = $vizinho;
        $tamanhoVisitados = $tamanhoVisitados + 1;

        $fila[$tamanhoFila] = $vizinho;
        $distancia[$tamanhoFila] = $distanciaAtual + 1;
        $tamanhoFila = $tamanhoFila + 1;
    }
}

```

Se a fila se esvaziar (inicioFila chegar a tamanhoFila) sem que \$personB tenha sido encontrada, o laço while termina naturalmente e a função devolve -1, indicando que não existe parentesco conhecido entre as duas pessoas.

Código completo do exercício 69

```
<?php

declare(strict_types=1);

function degreeOfSeparation(array $familyTree, string $personA, string $personB): int
{
    $fila = [];
    $distancia = [];
    $visitados = [];

    $tamanhoFila = 0;
    $inicioFila = 0;
    $tamanhoVisitados = 0;

    $fila[0] = $personA;
    $distancia[0] = 0;
    $tamanhoFila = 1;

    $visitados[0] = $personA;
    $tamanhoVisitados = 1;

    while ($inicioFila < $tamanhoFila) {
        $atual = $fila[$inicioFila];
        $distanciaAtual = $distancia[$inicioFila];
        $inicioFila = $inicioFila + 1;

        if ($atual == $personB) {
            return $distanciaAtual;
        }

        foreach ($familyTree as $pai => $filhos) {
            $vizinhos = [];

            if ($pai == $atual) {
                $vizinhos = array_merge($vizinhos, $filhos);
            }

            if (in_array($atual, $filhos)) {
                $vizinhos[] = $pai;

                foreach ($filhos as $irmao) {
                    if ($irmao != $atual) {
                        $vizinhos[] = $irmao;
                    }
                }
            }
        }

        foreach ($vizinhos as $vizinho) {
            $jaVisitado = false;
            for ($j = 0; $j < $tamanhoVisitados; $j++) {
                if ($visitados[$j] == $vizinho) {
                    $jaVisitado = true;
                }
            }

            if ($jaVisitado == false) {
                $visitados[$tamanhoVisitados] = $vizinho;
                $tamanhoVisitados = $tamanhoVisitados + 1;

                $fila[$tamanhoFila] = $vizinho;
                $distancia[$tamanhoFila] = $distanciaAtual + 1;
            }
        }
    }
}
```

```
        $tamanhoFila = $tamanhoFila + 1;
    }
}
return -1;
}
```